

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И
МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена трудового Красного Знамени федеральное государственное
бюджетное
образовательное учреждение высшего образования
«Московский технический университет связи и информатики»**

Кафедра Математическая кибернетика и информационные технологии

**Лабораторная работа 2
Инструмент сборки Gradle
по дисциплине
«Информационные технологии и программирование»**

Выполнил: студент гр. БВТ2403
Толибов Р.Ш.

Руководитель:

Москва, 2026 г.

Цель работы

Изучить инструмент автоматизации сборки Gradle, получить практические навыки создания и настройки Gradle-проекта, управления зависимостями, написания пользовательских задач, создания fat JAR и многомодульных проектов.

Задачи

- Создать базовый Gradle-проект и изучить структуру файлов `build.gradle.kts` и `settings.gradle.kts`
- Добавить зависимости Apache Commons Lang3, SLF4J и Logback
- Написать класс `Main` с логированием и обработкой строк
- Настроить плагин Shadow для создания fat JAR
- Создать пользовательские Gradle-задачи, в том числе генерацию build-паспорта
- Реализовать многомодульную структуру проекта
- Расширить задачу генерации паспорта хешем Git-коммита и счётчиком сборок

Ход работы

1. Создание проекта

Создан новый проект через IDE с Build System = Gradle, DSL = Kotlin. Структура проекта:

Была создана стандартная структура Maven-проекта:

```
Test-Gradle/
├── build.gradle.kts
├── settings.gradle.kts
└── src/
    └── main/
        ├── java/org/example/Main.java
        └── resources/
```

settings.gradle.kts:

```
plugins {
    id("org.gradle.toolchains.foojay-resolver-convention") version "1.0.0"
}
rootProject.name = "gradle"
include("app")
```

2. Зависимости и класс Main

build.gradle.kts (итоговый, со всеми плагинами):

```
plugins {
    id("java")
    application
    id("com.github.johnrengelman.shadow") version "8.1.1"
}

group = "org.example"
version = "1.0-SNAPSHOT"

application {
    mainClass.set("org.example.Main")
}

repositories {
    mavenCentral()
}
```

```

dependencies {
    implementation(project(":string-utils"))
    implementation("org.apache.commons:commons-lang3:3.14.0")
    implementation("org.slf4j:slf4j-api:2.0.9")
    implementation("ch.qos.logback:logback-classic:1.4.14")
    testImplementation(platform("org.junit:junit-bom:5.9.1"))
    testImplementation("org.junit.jupiter:junit-jupiter")
}

tasks.test {
    useJUnitPlatform()
}

tasks.shadowJar {
    manifest {
        attributes(Pair("Main-Class", "org.example.Main"))
    }
}

abstract class PrintInfoTask : DefaultTask() {
    @org.gradle.api.tasks.TaskAction
    fun print() {
        println("=====")
        println("Это моя первая пользовательская задача!")
        println("Проект: ${project.name}")
        println("Версия Gradle: ${project.gradle.gradleVersion}")
        println("=====")
    }
}

tasks.register<PrintInfoTask>("printInfo") {
    group = "Custom"
    description = "Выводит информацию о проекте"
}

abstract class GenerateBuildPassportTask : DefaultTask() {

    @get:org.gradle.api.tasks.Input
    abstract val gitCommitHash: Property<String>

    @org.gradle.api.tasks.TaskAction
    fun generate() {
        val resourcesDir = File(project.projectDir, "src/main/resources")
        resourcesDir.mkdirs()
        val outputFile = File(resourcesDir, "build-passport.properties")

        var buildNumber = 1
    }
}

```

```

        if (outputFile.exists()) {
            outputFile.readlines()
                .find { it.startsWith("build.number=") }
                ?.removePrefix("build.number=")
                ?.trim()
                ?.toIntOrNull()
                ?.let { buildNumber = it + 1 }
        }
        val user = System.getenv("USERNAME") ?: System.getenv("USER") ?:
"unknown"
        val os = System.getProperty("os.name")
        val javaVersion = System.getProperty("java.version")
        val now = java.time.LocalDateTime.now()
            .format(java.time.format.DateTimeFormatter.ofPattern("yyyy-MM-dd
HH:mm:ss"))
        outputFile.writeText("""
            build.number=$buildNumber
            build.user=$user
            build.os=$os
            build.java=$javaVersion
            build.time=$now
            build.git.hash=${gitCommitHash.get()}
            build.message=Assembled with Gradle. Have a nice day!
        """).trimIndent()

        println("Build passport generated: $outputFile (build #${buildNumber}")
    }
}

tasks.register<GenerateBuildPassportTask>("generateBuildInfo") {
    group = "Custom"
    description = "Генерирует build-passport.properties"

    // Получаем хеш последнего git-коммита
    val hash = try {
        val proc = ProcessBuilder("git", "rev-parse", "--short", "HEAD")
            .directory(project.projectDir)
            .start()
        proc.inputStream.bufferedReader().readLine()?.trim() ?: "unknown"
    } catch (e: Exception) {
        "unknown"
    }
    gitCommitHash.set(hash)
}

tasks.named("processResources") {
    dependsOn(tasks.named("generateBuildInfo"))
}

```

Main.java:

```

package gradle;

import java.util.Scanner;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.apache.commons.lang3.StringUtils;
public class Main {

    private static final Logger logger = LoggerFactory.getLogger(Main.class);

    public static void main(String[] args) {
        logger.info("Program started");

        Scanner sc = new Scanner(System.in);
        String name;
        if (sc.hasNextLine()) {
            name = StringUtils.capitalize(sc.nextLine());
        } else {
            name = "Ruslan";
        }
        logger.info("Hello, " + name + "!");
        logger.info("Program finished");
        sc.close();

        try {
            java.io.InputStream is = Main.class
                .getClassLoader()
                .getResourceAsStream("build-passport.properties");

            if (is != null) {
                java.util.Properties props = new java.util.Properties();
                props.load(is);

                logger.info("=== Build Passport ===");
                logger.info("User: " + props.getProperty("user"));
                logger.info("OS: " + props.getProperty("os"));
                logger.info("Java: " + props.getProperty("java.version"));
                logger.info("Message: " + props.getProperty("message"));
            } else {
                logger.warn("build-passport.properties not found!");
            }
        } catch (Exception e) {
            logger.error("Error reading build passport", e);
        }
    }
}

```

3. Fat JAR с плагином Shadow

Плагин добавлен в `plugins {}`, конфигурация `tasks.shadowJar` указывает `Main-Class`. Сборка и запуск:

```
gradle shadowJar
java -jar build/libs/Test-Gradle-1.0-SNAPSHOT-all.jar
```

Fat JAR включает все зависимости (Commons Lang3, SLF4J, Logback) — программа запускается без дополнительного `classpath`.

4. Пользовательские задачи

printInfo — выводит имя проекта и версию Gradle:

```
private int hash(K key) {
    return Math.abs(key.hashCode()) % capacity; // BAD
}
```

generateBuildInfo — генерирует `build-passport.properties` в `src/main/resources/`. Задача настроена как зависимость для `processResources`, поэтому файл автоматически попадает в JAR при каждом `gradle build` или `gradle run`.

Вывод

В ходе лабораторной работы был изучен инструмент сборки Gradle и его ключевые возможности. Был создан проект с Kotlin DSL, подключены внешние зависимости (Apache Commons Lang3, SLF4J/Logback, Jackson), настроен плагин `application` для указания главного класса и плагин `Shadow` для сборки исполняемого fat JAR со всеми зависимостями внутри.

Были написаны пользовательские задачи на Kotlin: `printInfo` для вывода информации о проекте и `generateBuildInfo` для автоматической генерации паспорта сборки. Задача `generateBuildInfo` интегрирована в жизненный цикл сборки через зависимость от `processResources`, что обеспечивает автоматическое включение паспорта в JAR при каждой сборке.

<https://github.com/TRuslan666/ITaP/tree/main/лаб2>